Data Structures

CPT102 – Lecture 8a

Erick Purwanto

Xi'an Jiaotong-Liverpool University

西交利物浦大学

CPT102 Data Structures

Lecture 8a

Union Find

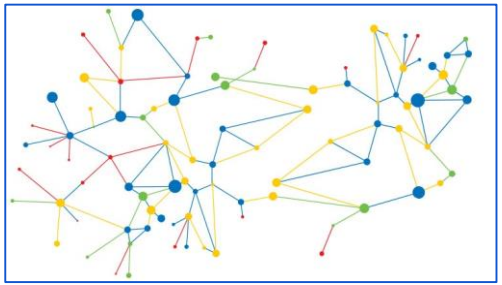
Welcome !

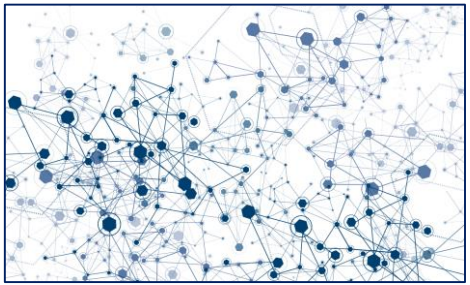
- Welcome to Lecture 8!
  - learn about how to use arrays in a smart way to implement two cool data structures: union find and heaps

- Let's start with Lecture 8a!
- In this lecture we are going to
  - learn about a sophisticated data structure called **Union Find**
  - also called *Disjoint Set* data structure

Union Find and Dynamic Connectivity

- In this lecture, we will explore a new data structure that uses a simple array as a building block
- We will build a data structure called the **Union Find**
  - to solve the **Dynamic Connectivity** problem
- Furthermore, we will see:
  - How a data structure design can evolve from *basic* to *sophisticated*
  - How our choice of *underlying abstraction* can affect the running time





Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)



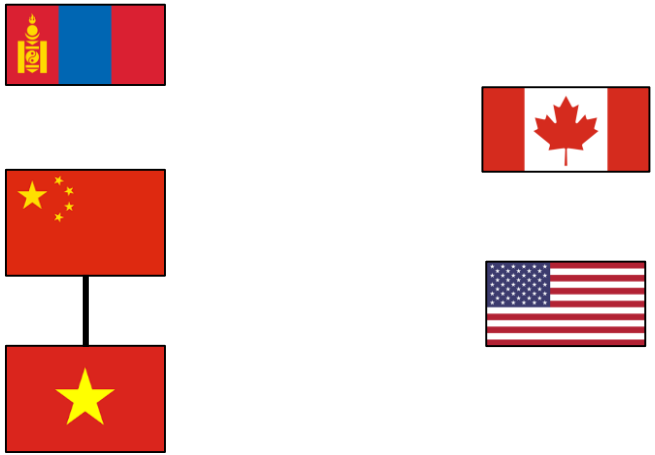


Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)

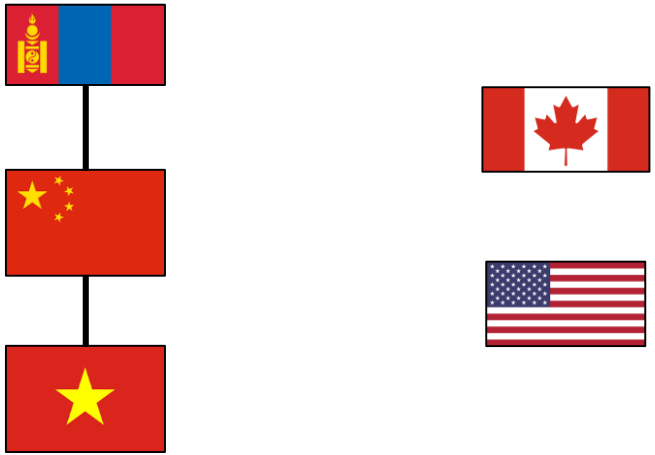


Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)
- isSameGroup(Vietnam, Mongolia) → ?

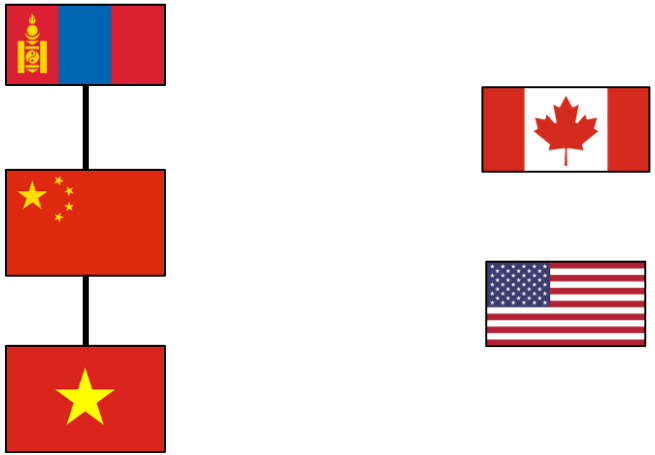


Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)
- isSameGroup(Vietnam, Mongolia) → `true`
- union(USA, Canada)

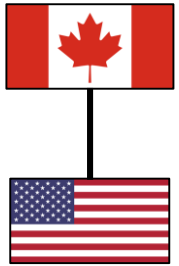


Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)
- isSameGroup(Vietnam, Mongolia) → `true`
- union(USA, Canada)
- isSameGroup(Canada, Mongolia) → ?

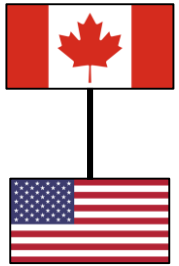


Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)
- isSameGroup(Vietnam, Mongolia) → `true`
- union(USA, Canada)
- isSameGroup(Canada, Mongolia) → `false`
- union(China, USA)

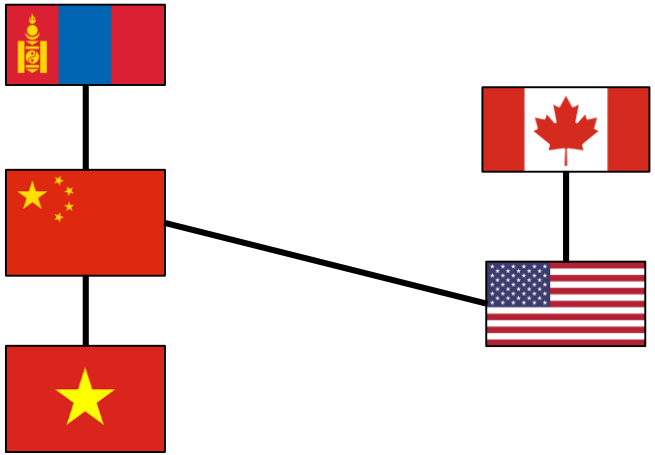


Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)
- isSameGroup(Vietnam, Mongolia) → `true`
- union(USA, Canada)
- isSameGroup(Canada, Mongolia) → `false`
- union(China, USA)
- isSameGroup(Canada, Mongolia) → ?



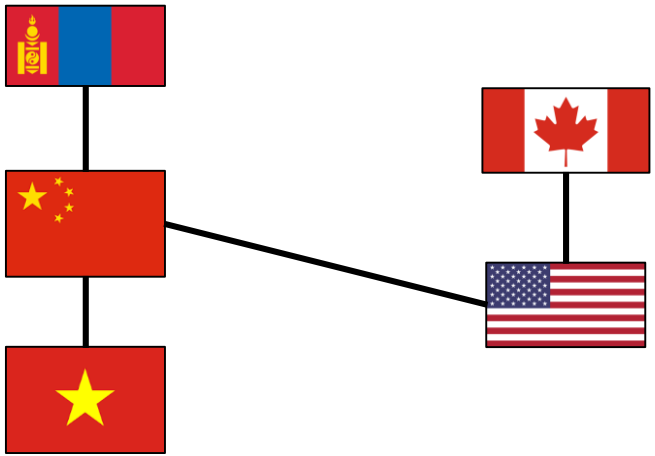
Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

Example:

- union(China, Vietnam)
- union(China, Mongolia)
- isSameGroup(Vietnam, Mongolia) → `true`
- union(USA, Canada)
- isSameGroup(Canada, Mongolia) → `false`
- union(China, USA)
- isSameGroup(Canada, Mongolia) → `true`

the answer can change!



Union Find

- Union Find data structure has two operations:
  - **union**(x, y) : Combines x and y
  - **isSameGroup**(x, y) : Returns **`true`** if x is in the same group as y

- This data structure is useful for, for example:
  - Percolation theory in Computational Chemistry
  - Kruskal Algorithm in computing MST
  - and many more

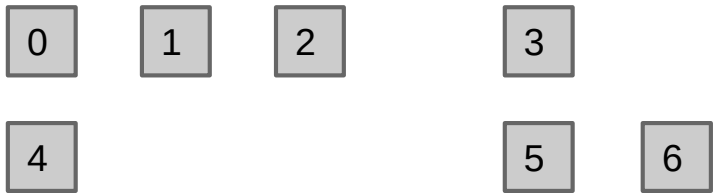
Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

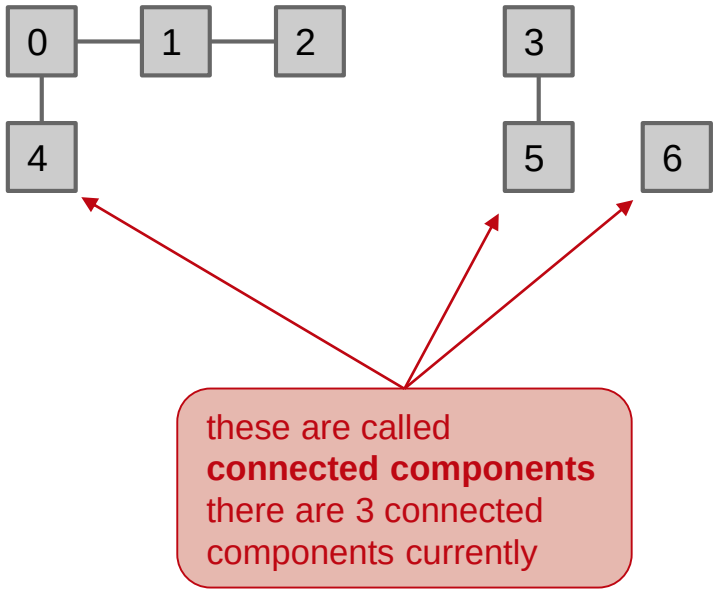
```
uf = UnionFind(7)
```



Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

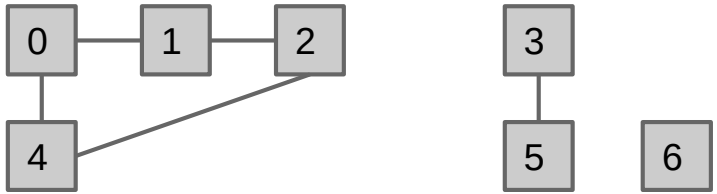
```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
uf.isSameGroup(2, 4) → true
uf.isSameGroup(3, 0) → false
uf.union(4, 2)
```



Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

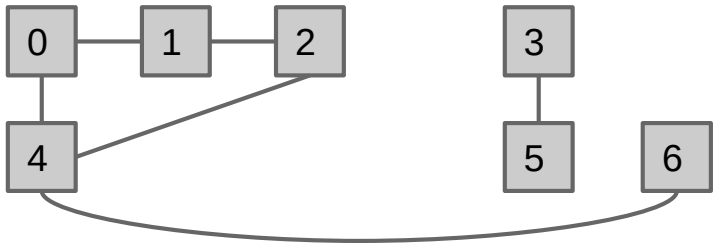
```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
uf.isSameGroup(2, 4) → true
uf.isSameGroup(3, 0) → false
uf.union(4, 2)
uf.union(4, 6)
```



Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

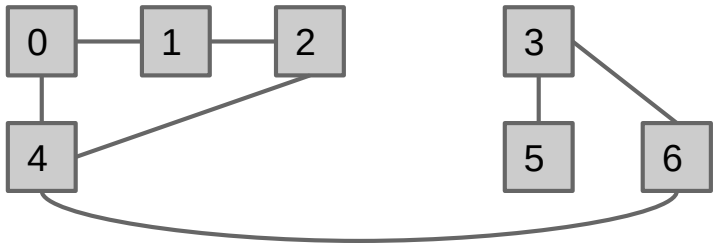
```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
uf.isSameGroup(2, 4) → true
uf.isSameGroup(3, 0) → false
uf.union(4, 2)
uf.union(4, 6)
uf.union(3, 6)
```



Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

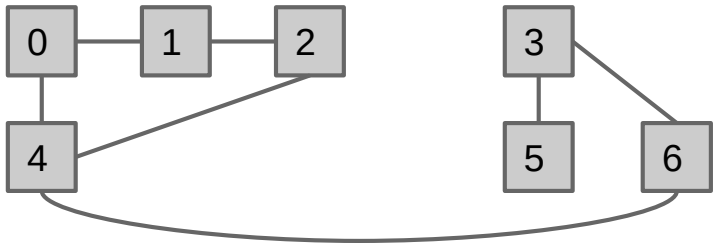
```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
uf.isSameGroup(2, 4) → true
uf.isSameGroup(3, 0) → false
uf.union(4, 2)
uf.union(4, 6)
uf.union(3, 6)
uf.isSameGroup(3, 0) → ?
```



Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
uf.isSameGroup(2, 4) → true
uf.isSameGroup(3, 0) → false
uf.union(4, 2)
uf.union(4, 6)
uf.union(3, 6)
uf.isSameGroup(3, 0) → true
```



Union Find on Integers

- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*
- We want to implement two methods: (and a constructor)

```
/** Combines two items p and q. */
void union(int p, int q) {
    ...
}
/** Decides if two items p and q are in the same group. */
boolean isSameGroup(int p, int q) {
    ...
}
```

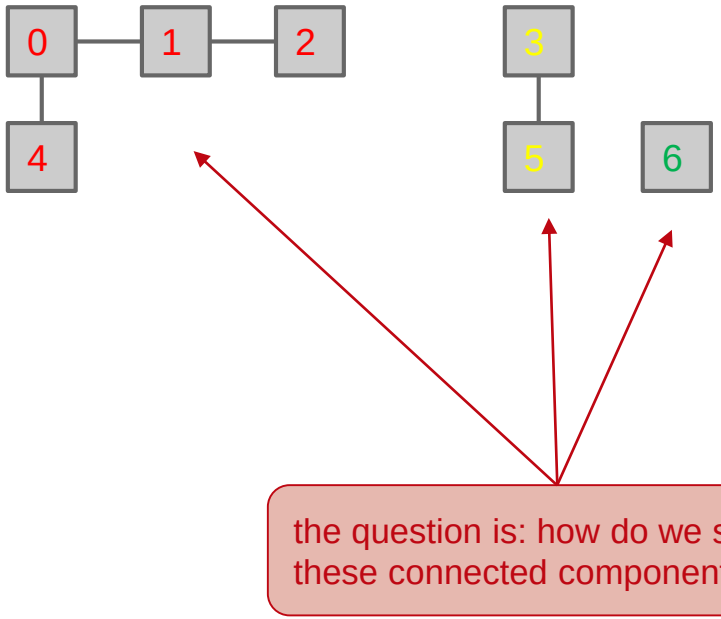
check **find(p) == find(q)**
find(p) → the group id of p

  - where the ***number of items*** **N** and the ***number of operations*** **M** can be *very large*
  - and the two methods are called in *arbitrary* order

Union Find on Integers

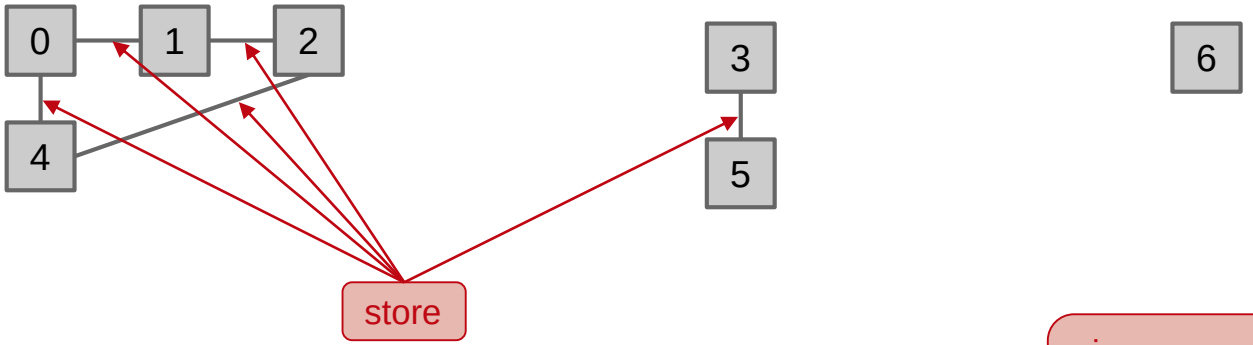
- Let's start simple, we let:
  - the items are integers instead of arbitrary data, and we
  - declare the number of items in advance, where everything is *disconnected*

```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
```



Naive Approach

- Let us start to think how we store the connectivity information
- A naive approach:
  - Union: Record every single connecting line in some data structure
  - isSameGroup: Iterate over the lines to see if one item can be reached from the other



since connectivity is transitive,
this idea is redundant
what should we store instead?

Connected Components / Sets

- A better idea would be to record **the sets** that each item belongs to
  - each set represents a connected component

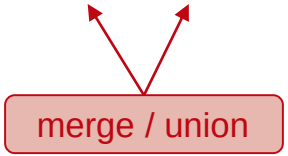
```
uf = UnionFind(7)        {0}, {1}, {2}, {3}, {4}, {5}, {6}
```

Connected Components / Sets

- A better idea would be to record **the sets** that each item belongs to
  - each set represents a connected component

```
uf = UnionFind(7)
uf.union(0, 1)
```

{0}, {1}, {2}, {3}, {4}, {5}, {6}

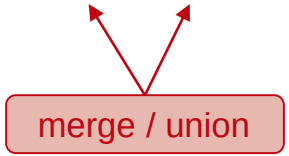


Connected Components / Sets

- A better idea would be to record **the sets** that each item belongs to
  - each set represents a connected component

```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
```

```
{0}, {1}, {2}, {3}, {4}, {5}, {6}
{0, 1}, {2}, {3}, {4}, {5}, {6}
```

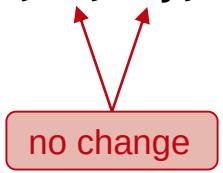


Connected Components / Sets

- A better idea would be to record **the sets** that each item belongs to
  - each set represents a connected component

```
uf = UnionFind(7)
uf.union(0, 1)
uf.union(1, 2)
uf.union(0, 4)
uf.union(3, 5)
uf.isSameGroup(2, 4)
uf.isSameGroup(3, 0)
uf.union(4, 2)
```

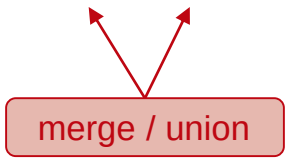
```
{0}, {1}, {2}, {3}, {4}, {5}, {6}
{0, 1}, {2}, {3}, {4}, {5}, {6}
{0, 1, 2}, {3}, {4}, {5}, {6}
{0, 1, 2, 4}, {3}, {5}, {6}
{0, 1, 2, 4}, {3, 5}, {6}
```



Connected Components / Sets

- A better idea would be to record **the sets** that each item belongs to
  - each set represents a connected component

```
uf = UnionFind(7)          {0}, {1}, {2}, {3}, {4}, {5}, {6}
uf.union(0, 1)             {0, 1}, {2}, {3}, {4}, {5}, {6}
uf.union(1, 2)             {0, 1, 2}, {3}, {4}, {5}, {6}
uf.union(0, 4)             {0, 1, 2, 4}, {3}, {5}, {6}
uf.union(3, 5)             {0, 1, 2, 4}, {3, 5}, {6}
uf.isSameGroup(2, 4)
uf.isSameGroup(3, 0)
uf.union(4, 2)             {0, 1, 2, 4}, {3, 5}, {6}
uf.union(4, 6)             {0, 1, 2, 4, 6}, {3, 5}
uf.union(3, 6)
```



Connected Components / Sets

- A better idea would be to record the sets that each item belongs to
  - each set represents a connected component
  - check whether two items are **in the same set**

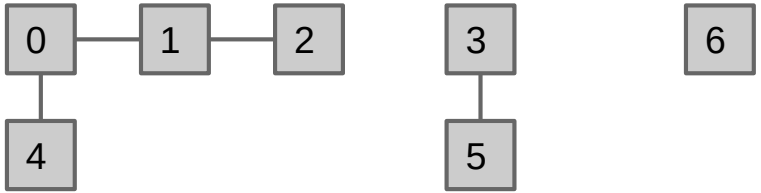
```
uf = UnionFind(7)          {0}, {1}, {2}, {3}, {4}, {5}, {6}
uf.union(0, 1)             {0, 1}, {2}, {3}, {4}, {5}, {6}
uf.union(1, 2)             {0, 1, 2}, {3}, {4}, {5}, {6}
uf.union(0, 4)             {0, 1, 2, 4}, {3}, {5}, {6}
uf.union(3, 5)             {0, 1, 2, 4}, {3, 5}, {6}
uf.isSameGroup(2, 4)
uf.isSameGroup(3, 0)
uf.union(4, 2)             {0, 1, 2, 4}, {3, 5}, {6}
uf.union(4, 6)             {0, 1, 2, 4, 6}, {3, 5}
uf.union(3, 6)             {0, 1, 2, 3, 4, 5, 6}
uf.isSameGroup(3, 0)
```

asking: are 3 and 6 in the same set ?

Selecting Underlying Data Structure

- So, what data structure should we should to track the set membership?
  - in other words, how do we keep track which item in which set?

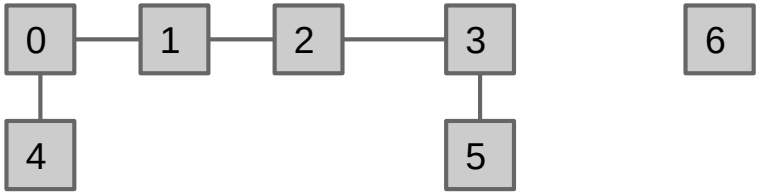
Before `union(2, 3)` operation:



`{0, 1, 2, 4}, {3, 5}, {6}`

5 in here before

After `union(2, 3)` operation:



`{0, 1, 2, 4, 3, 5}, {6}`

5 in here after

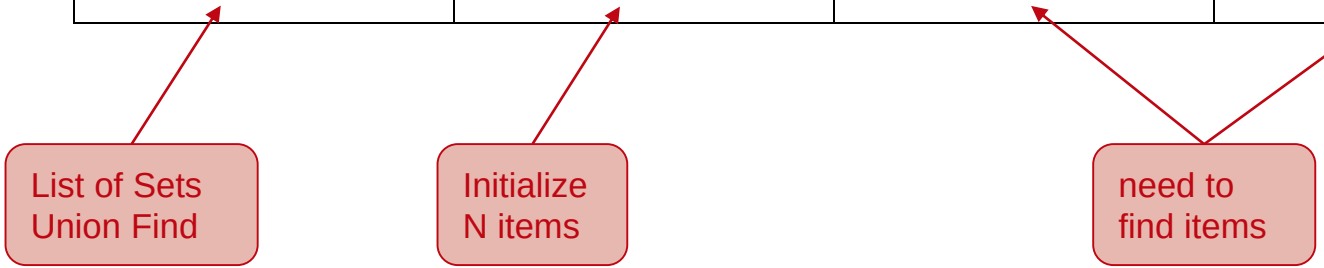
Idea 1: List of Sets

- Using `List<Set<Integer>>` from Java Library
  - A very intuitive idea
  - However, imagine in the beginning, we have this List of Sets of integers: `[{0}, {1}, {2}, {3}, {4}, {5}, {6}]`
  - We have to iterate through all the sets to find an item
    - complicated and slow!
  - Worst case: if nothing is combined, then `isSameGroup(5, 6)` requires iterating through N-1 sets to find 5, then N sets to find 6
    - overall runtime is linear or O(N)
    - similarly for `union`

Performance Summary

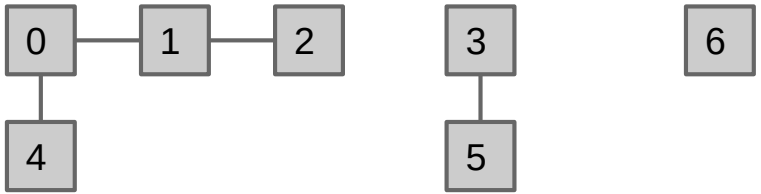
- Let's keep track our progress so far

Implementation	constructor	union	isSameGroup
ListOfSetsUF	O(N)	O(N)	O(N)



Idea 2: Array of Integers (1)

- Using an array of integers, where the ith entry gives ***the set number/id*** of item i



{0, 1, 2, 4}, {3, 5}, {6}

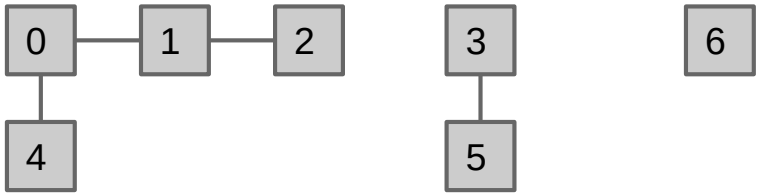
int[] id	4	4	4	5	4	5	6
	0	1	2	3	4	5	6

item 0, 1, 2, and 4 belong to set number 4

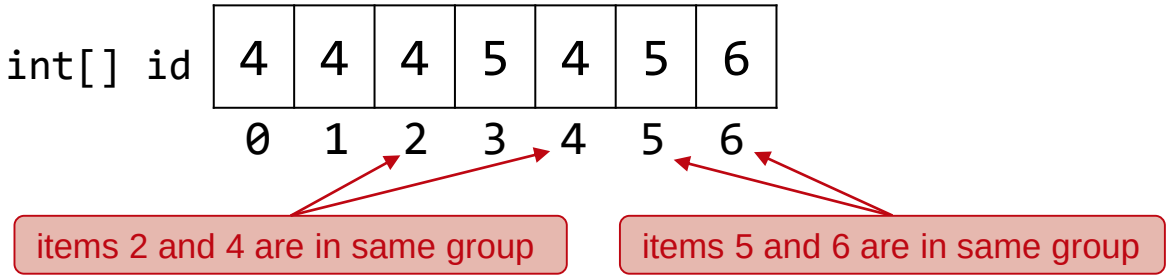
the set number 4 does not matter, could be any int

Idea 2: Array of Integers (2)

- Using an array of integers, where the ith entry gives the set number/id of item i
  - **isSameGroup**(p, q) simply checks whether id[p] and id[q] are ***the same***



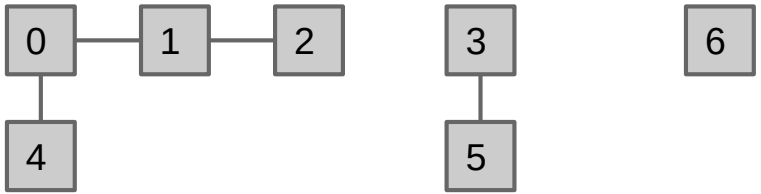
{0, 1, 2, 4}, {3, 5}, {6}



Idea 2: Array of Integers (3)

- Using an array of integers, where the ith entry gives the set number/id of item i
  - **union**(p, q) simply changes **all** entries that equal id[p] to id[q]

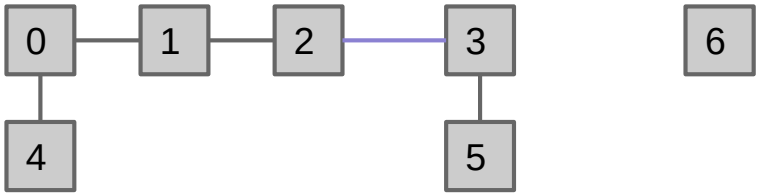
Before union(2, 3) operation:



{0, 1, 2, 4}, {3, 5}, {6}

int[] id	4	4	4	5	4	5	6
	0	1	2	3	4	5	6

After union(2, 3) operation:



{0, 1, 2, 4, 3, 5}, {6}

int[] id	5	5	5	5	5	5	6
	0	1	2	3	4	5	6

the set number of 0, 1, 2, 4 changes from 4 to 5, which is the set number of 3

Idea 2: Array of Integers (4)

- This implementation of Union Find data structure is called **`QuickFind`**

```java
public class QuickFind {

    private int[] id;

    /** Construct a new union find data structure of N elements. */
    public QuickFind(int N){
        id = new int[N];
        for (int i = 0; i < N; i++){
            id[i] = i;
        }
    }

    /** Combines two elements p and q. */
    public void union(int p, int q){
        int pid = id[p];
        int qid = id[q];
        for (int i = 0; i < id.length; i++){
            if (id[i] == pid){
                id[i] = qid;
            }
        }
    }

    /** Decides if two elements p and q are in the same group. */
    public boolean isSameGroup(int p, int q){
        return (id[p] == id[q]);
    }
}
```

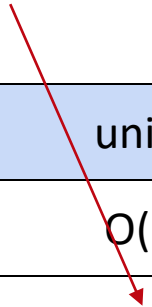
because the **isSameGroup/find** operation is ***very fast***, just two array accesses

initially, each item belong to each set

however, as seen in the for loop here, **union** is ***slow***, taking O(N) time

Performance Summary

- Our second attempt made some progress with `isSameGroup`,
  - but connecting two items taking linear time each time is unacceptable!

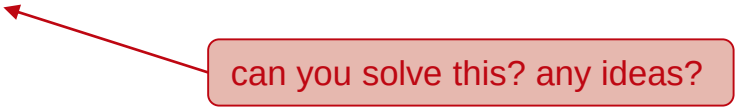


Implementation	constructor	union	isSameGroup
ListOfSetsUF	O(N)	O(N)	O(N)
QuickFind	O(N)	O(N)	O(1)

Next Idea

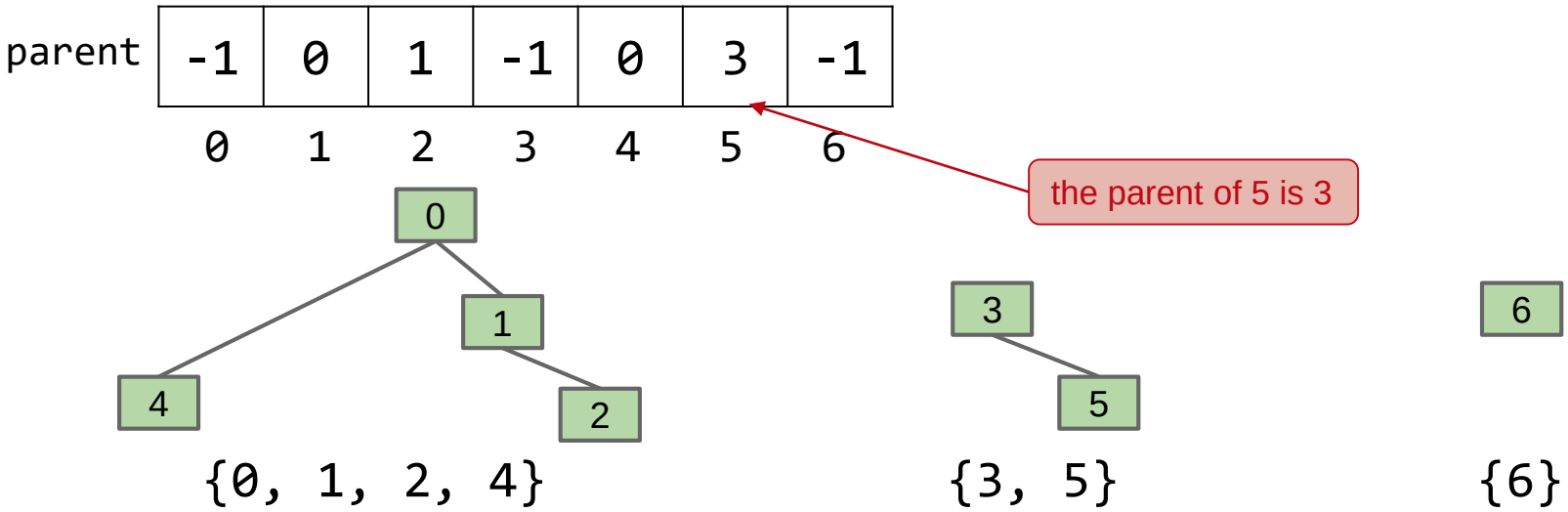
- Because we may need to change *many values*, `union` of `QuickFind` is slow
  - we still want to represent the connected components as an array of integers
  - however, values will be chosen so that union is fast

- How could we change our connected components representation so that combining two sets requires changing only **one value**?



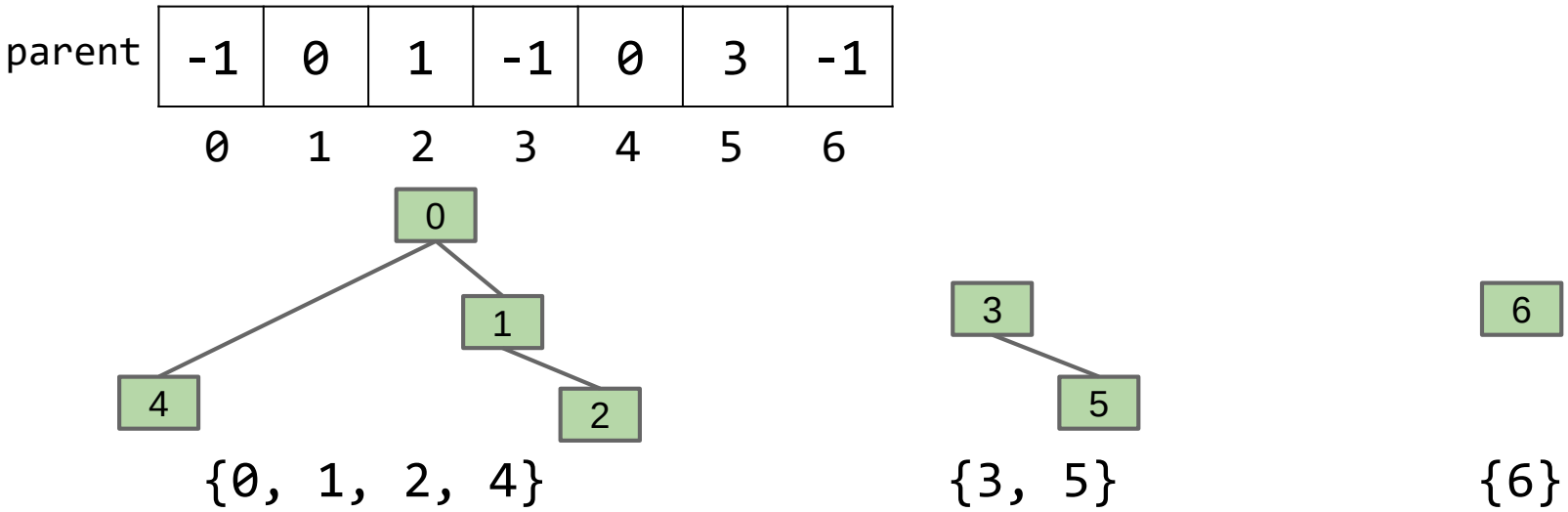
Idea 3: Array of Parents

- How could we change our set representation so that combining two sets requires changing only **one value**?
  - Assign each item a parent (instead of an id), and set the roots to have value -1
  - Resulting in a forest (a set of trees)



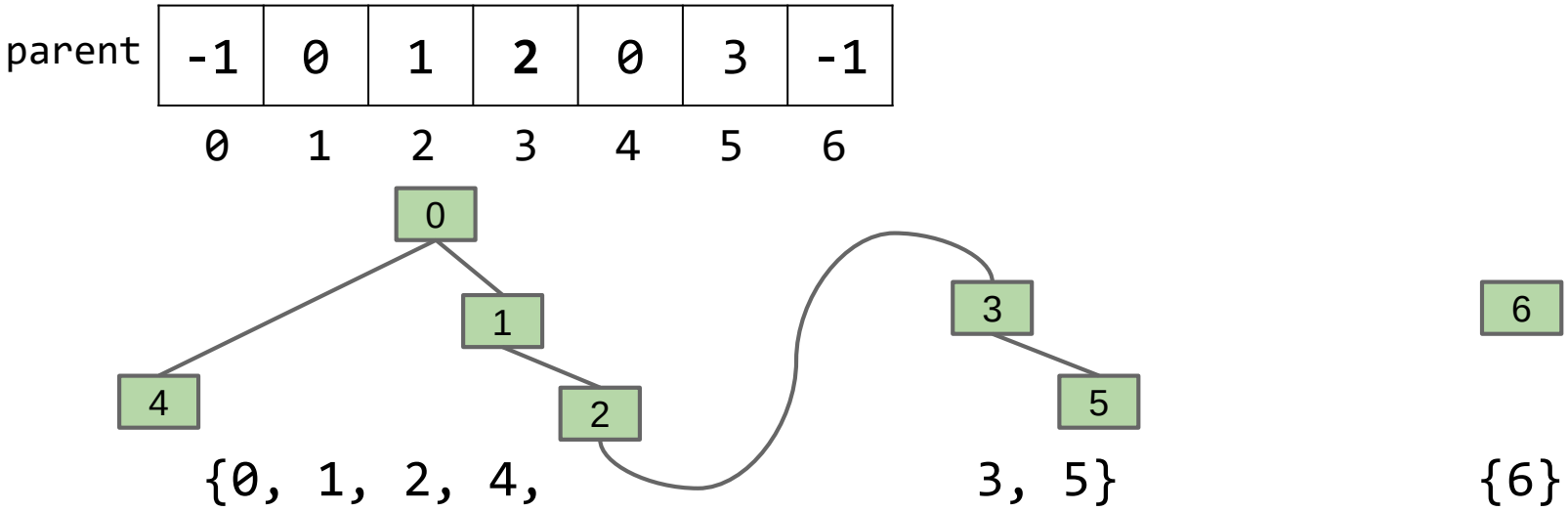
Improving Union (1)

- How should we change the parent array if we call **union(5, 2)**?
  - Can we just set parent[5] to 2?



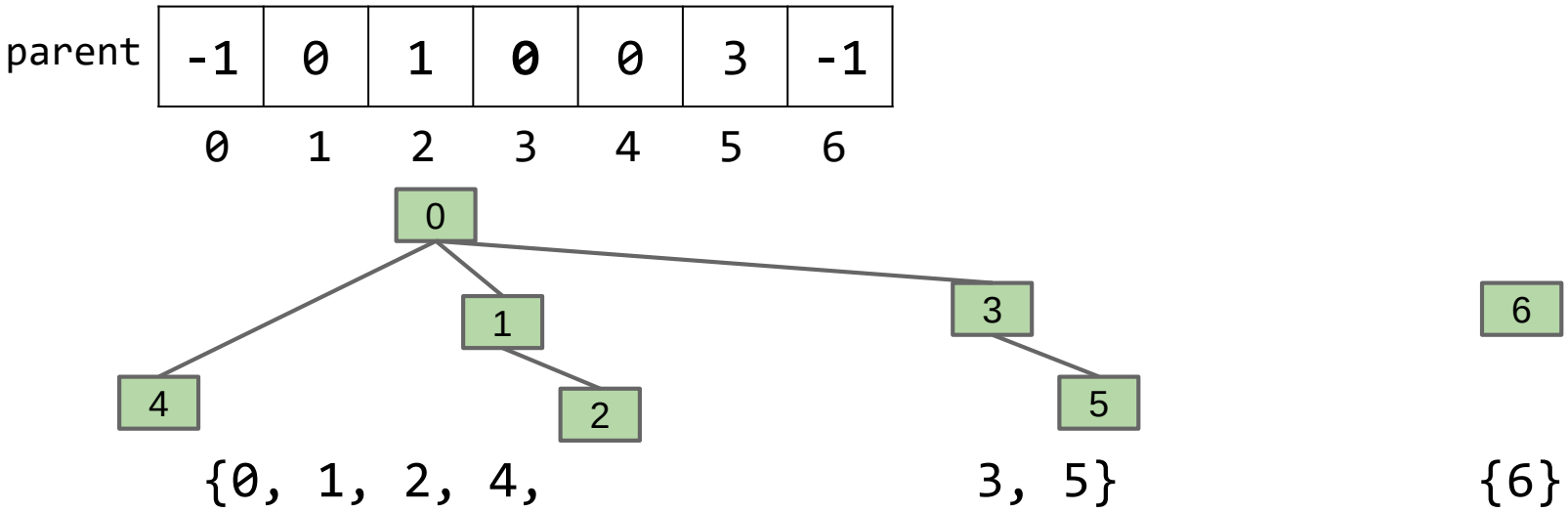
Improving Union (2)

- How should we change the parent array if we call **union(5, 2)**?
  - Can we just set parent[5] to 2? No, 3, the root of 5, will be disconnected
  - One possible idea is to set parent[3] to 2, but this results in taller tree, longer path to root



Improving Union (3)

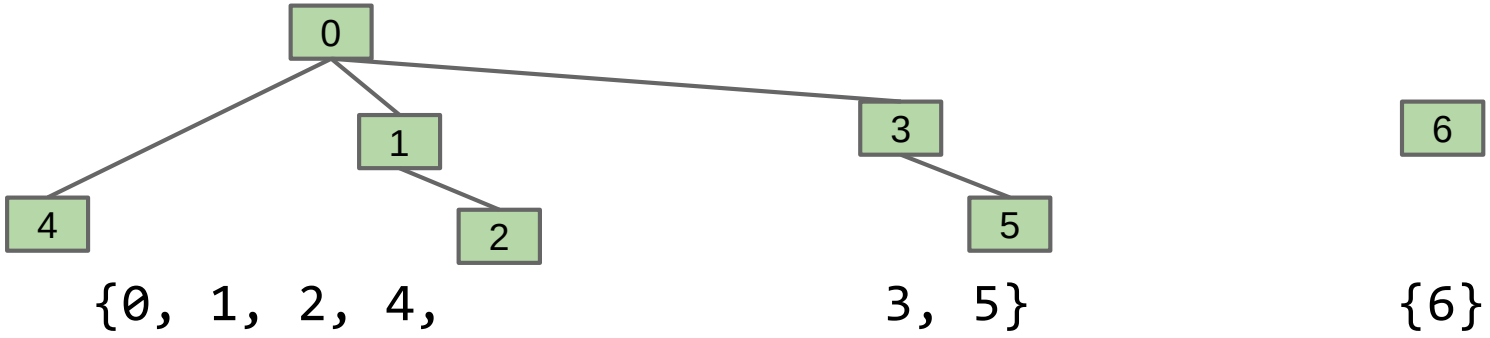
- How should we change the parent array if we call **union(5, 2)**?
  - Another idea is to do the following:
    find the root 5, find the root of 2,
    and then set the value of the root of 5 to be the root of 2



Improving Union (3)

- How should we change the parent array if we call **union(5, 2)**?
  - Another idea is to do the following:
    find the root 5, find the root of 2,
    and then set the value of the root of 5 to be the root of 2

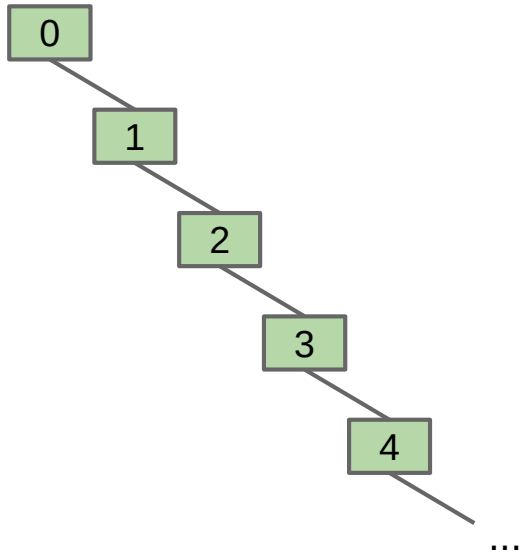
parent	-1	0	1	0	0	3	-1
	0	1	2	3	4	5	6



seems like a good idea…
can things go wrong?

The Worst-Case

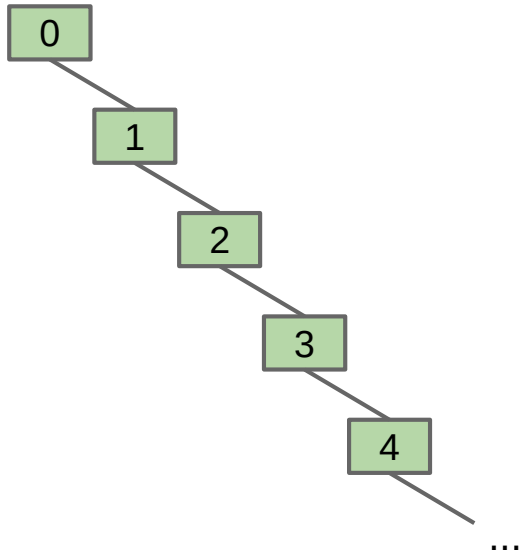
- Consider the following series of operations:
  - ...
  - union(4, 3)
  - union(3, 2)
  - union(2, 1)
  - union(1, 0)



- For N items, what is the worst case runtime of:
  - union(p, q)?
  - isSameGroup(p, q)?

The Worst-Case

- Consider the following series of operations:
  - ...
  - union(4, 3)
  - union(3, 2)
  - union(2, 1)
  - union(1, 0)



- For N items, what is the worst case runtime of:
  - union(p, q)? O(N)
  - isSameGroup(p, q)? O(N)

Idea 3: Array of Parents

- This implementation of Union Find data structure is called **`QuickUnion`**

```java
public class QuickUnion {

    private int[] parent;

    /** Construct a new union find data structure of N elements. */
    public QuickUnion(int N) {
        parent = new int[N];
        for (int i = 0; i < N; i++) {
            parent[i] = -1;
        }
    }

    // Finding the root.
    private int find(int p) {
        while (parent[p] >= 0) {
            p = parent[p];
        }
        return p;
    }

    /** Combines two elements p and q. */
    public void union(int p, int q) {
        int i = find(p);
        int j = find(q);
        parent[i] = j;
    }

    /** Decides if two elements p and q are in the same group. */
    public boolean isSameGroup(int p, int q) {
        return find(p) == find(q);
    }
}
```

finding the root costs O(N) in worst-case

causing union and isSameGroup to cost O(N) worst-case running time as well

Performance Summary

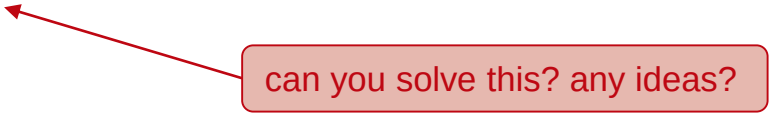
- Our third attempt may perform well in average,
  but in the worst case, we actually get a worse performance!

Implementation	Constructor	union	isSameGroup
ListOfSetsUF	O(N)	O(N)	O(N)
QuickFind	O(N)	O(N)	O(1)
QuickUnion	O(N)	O(N)	O(N)

Next Idea

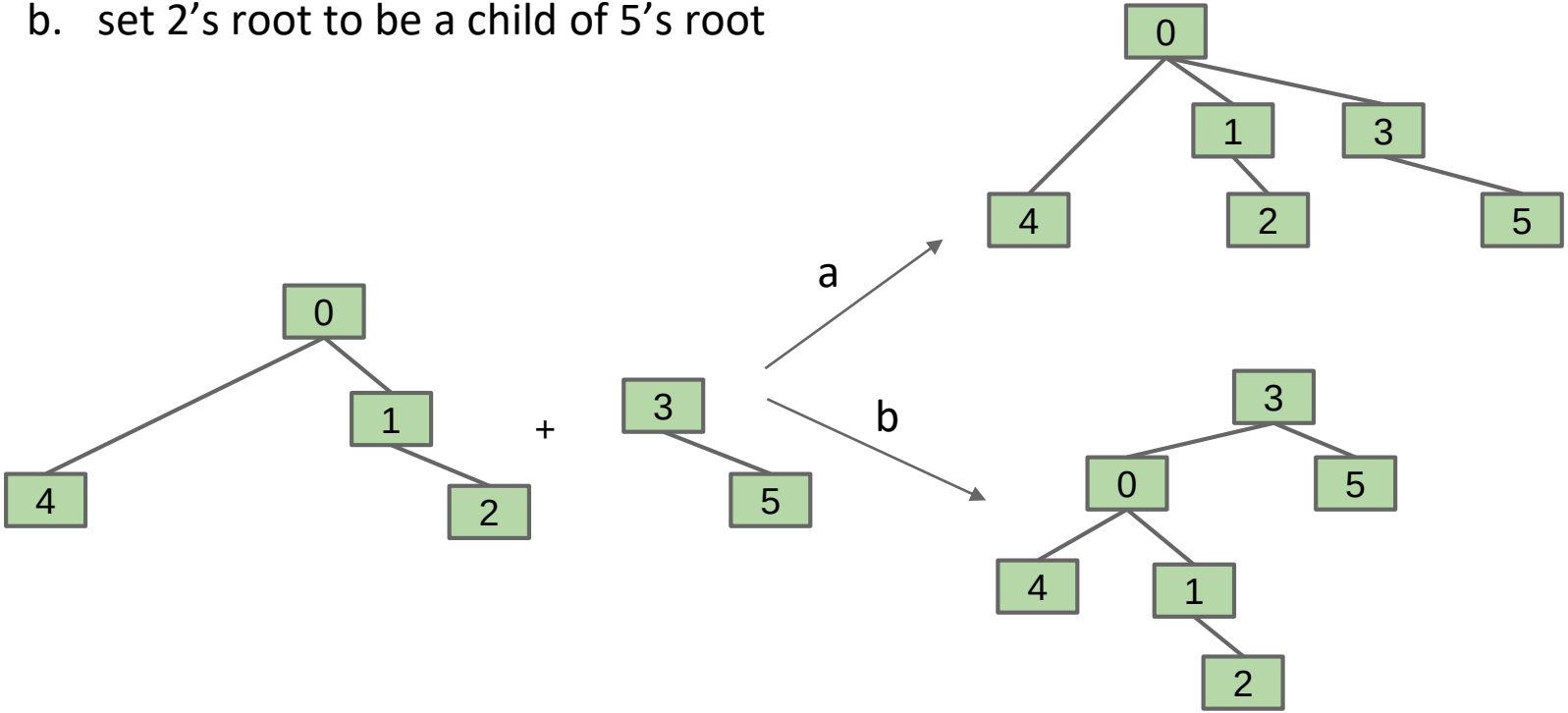
- The defect of `QuickUnion` is that the trees can get tall
  - resulting in potentially even worse performance than `QuickFind` if tree is imbalanced

- What should we do to keep our trees balanced?



Better Union

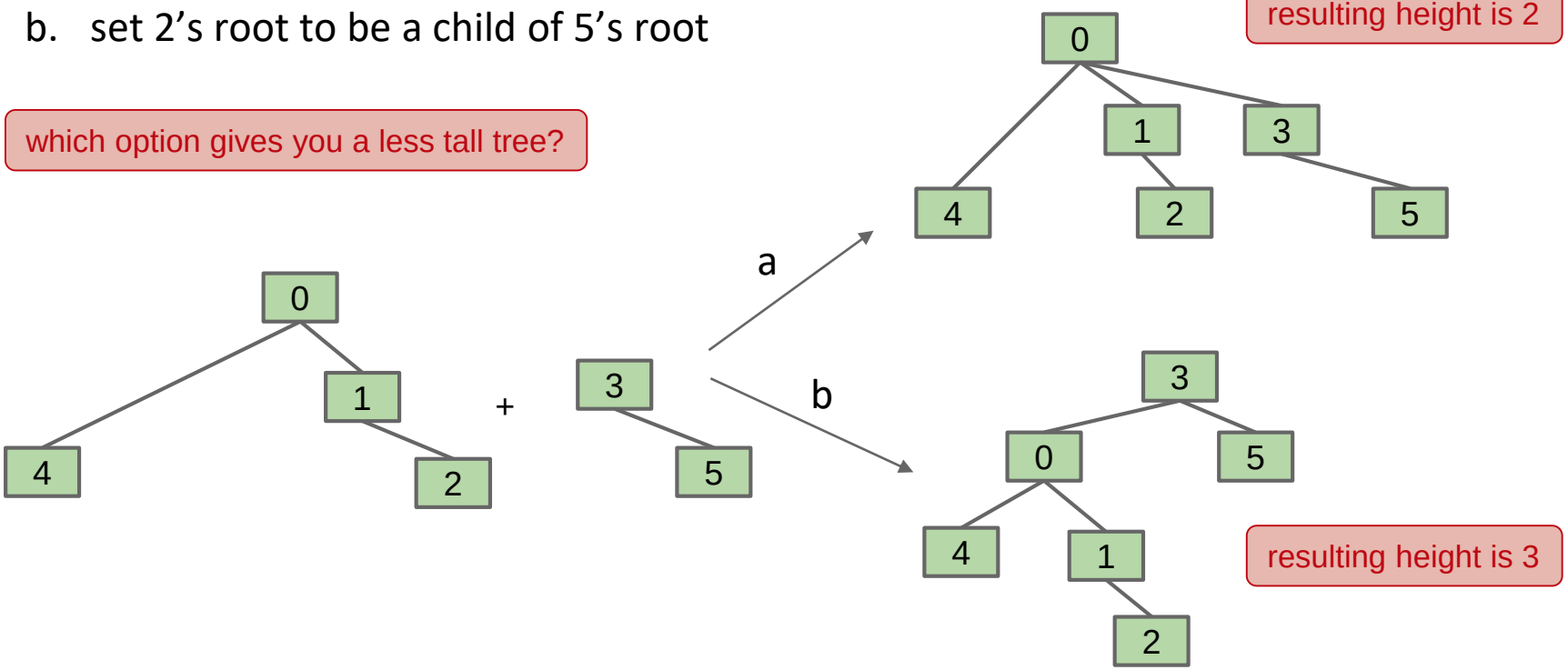
- Suppose you are trying to union(5, 2) — which one is better :
  a. set 5's root to be a child of 2's root
  b. set 2's root to be a child of 5's root



An Observation

- Suppose you are trying to union(5, 2) — which one is better :
  a. set 5's root to be a child of 2's root
  b. set 2's root to be a child of 5's root

which option gives you a less tall tree?



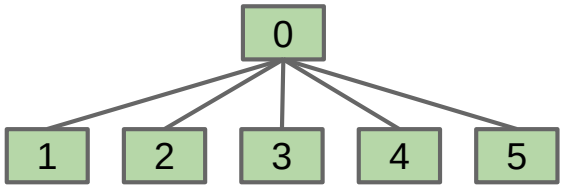
Idea 4: Link Smaller to Larger

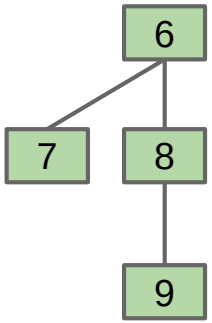
- Rather than always links the root of *first* tree to the root of *second* tree during union as in QuickUnion:
  - keep track of the **tree size** (its number of elements)
  - link the the root of ***smaller*** tree to the root of ***larger*** tree

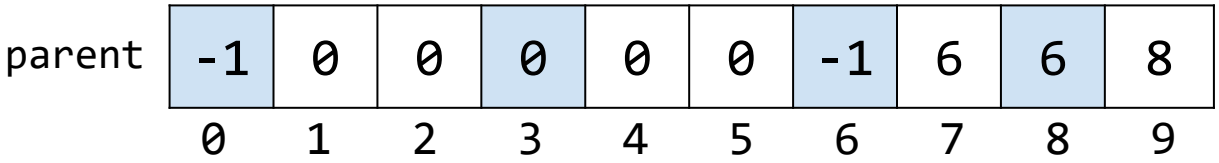
Example Linking Smaller to Larger (1)

- Rather than always links the root of *first* tree to the root of *second* tree during union as in QuickUnion:
  - keep track of the **tree size** (its number of elements)
  - link the the root of ***smaller*** tree to the root of ***larger*** tree

- Example: what will union(3, 8) change ?



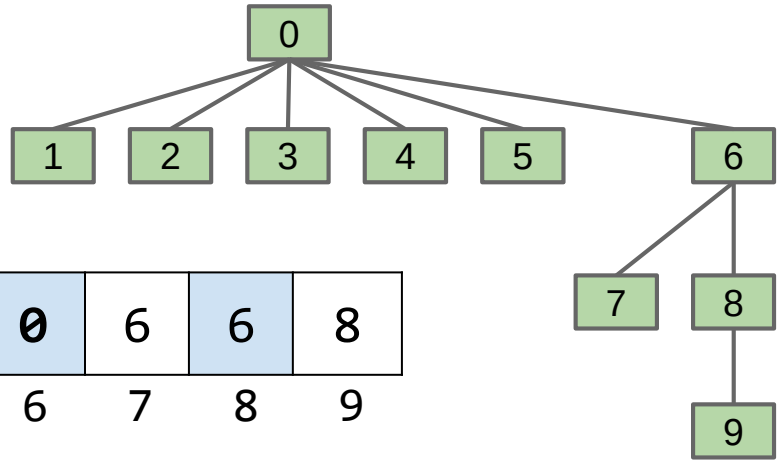




Example Linking Smaller to Larger (2)

- Rather than always links the root of *first* tree to the root of *second* tree during union as in QuickUnion:
  - keep track of the **tree size** (its number of elements)
  - link the the root of ***smaller*** tree to the root of ***larger*** tree

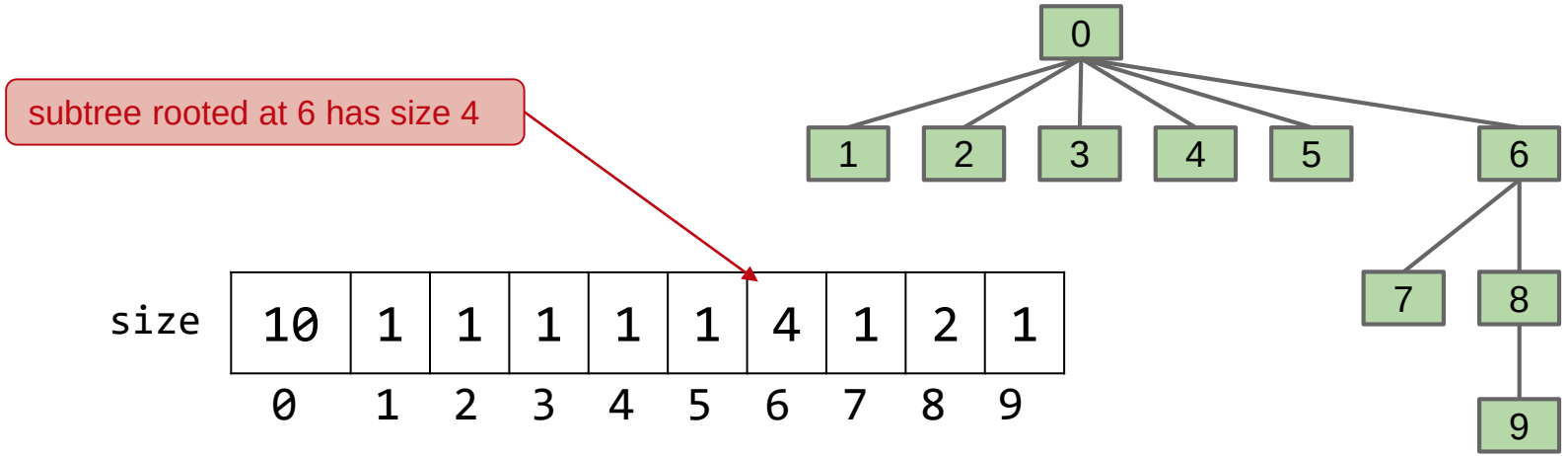
- Example: what will union(3, 8) change ?
  - the tree of 8 has smaller size,
    so we change the root of 6



	0	1	2	3	4	5	6	7	8	9
parent	-1	0	0	0	0	0	0	6	6	8

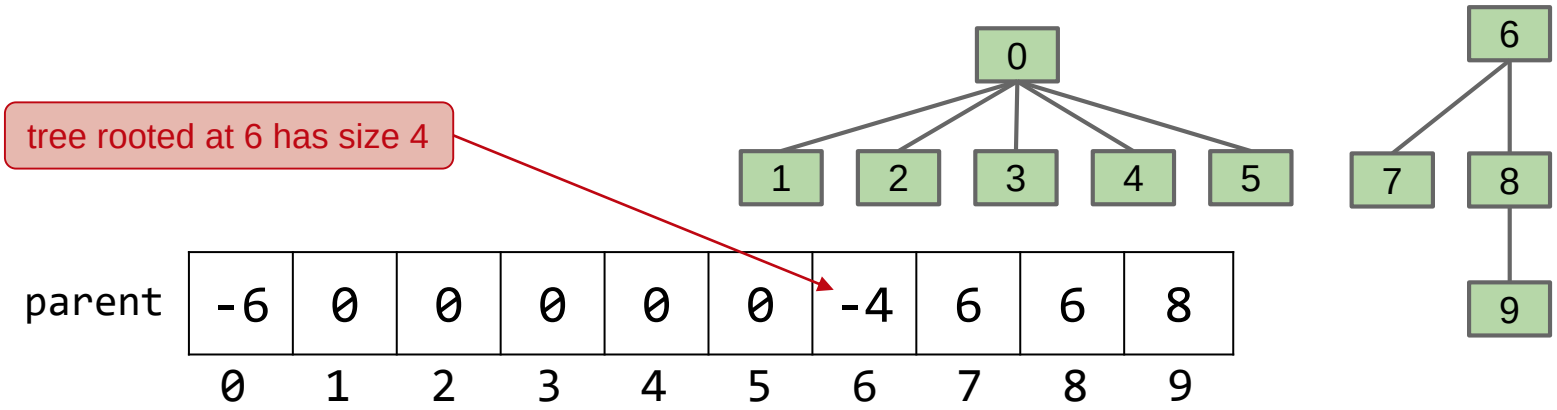
Idea 4.1: Weighted Quick Union (1)

- How do we implement this? What is the underlying data structure(s)?
  - we still use array of parents as in QuickUnion
  - to keep track the sizes:
    1. we could have another array size[] to store ***the size of the subtree rooted at index i***



Idea 4.2: Weighted Quick Union (2)

- How do we implement this? What is the underlying data structure(s)?
  - we still use array of parents as in QuickUnion
  - to keep track the sizes:
    1. we could have another array size[] to store the size of the subtree rooted at index i
    2. instead of -1, we store ***negative size*** of the trees in the root



Relationship between Weight and Height (1)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

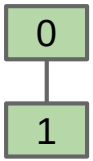
N	H
1	0

when N = 1, the tallest possible tree has height H = 0

Relationship between Weight and Height (2)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

N	H
1	0
2	1

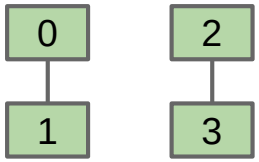


when N = 2, the tallest possible tree has height H = 1

Relationship between Weight and Height (3)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

N	H
1	0
2	1

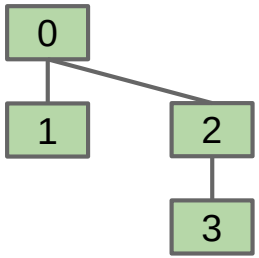


we will union those two trees next

Relationship between Weight and Height (4)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

N	H
1	0
2	1
4	2

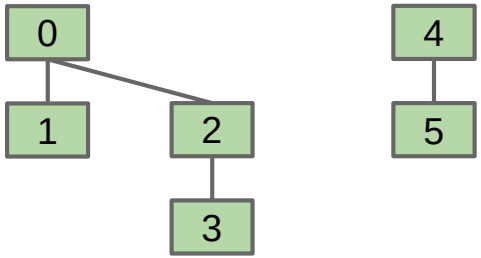


when N = 4, the tallest possible tree has height H = 2

Relationship between Weight and Height (5)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

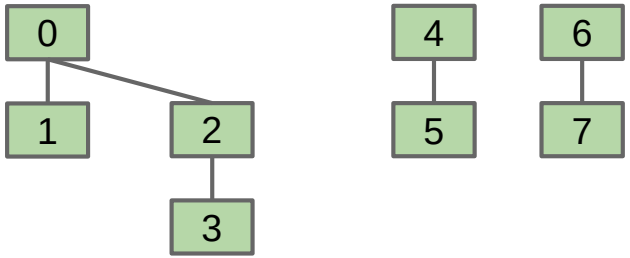
N	H
1	0
2	1
4	2



Relationship between Weight and Height (6)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

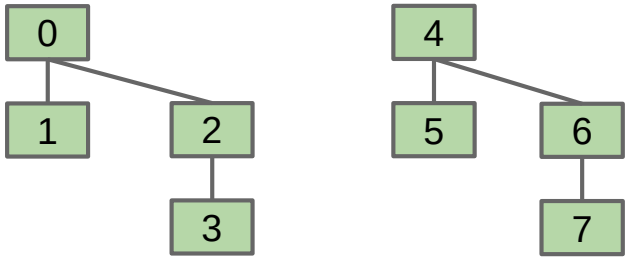
N	H
1	0
2	1
4	2



Relationship between Weight and Height (7)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

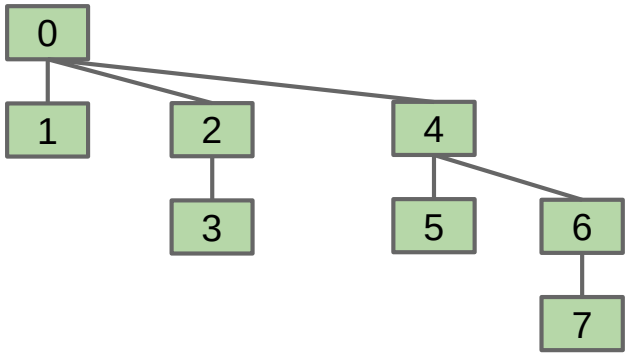
N	H
1	0
2	1
4	2



union those two to get a height-3 tree

Relationship between Weight and Height (8)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)



N	H
1	0
2	1
4	2
8	3

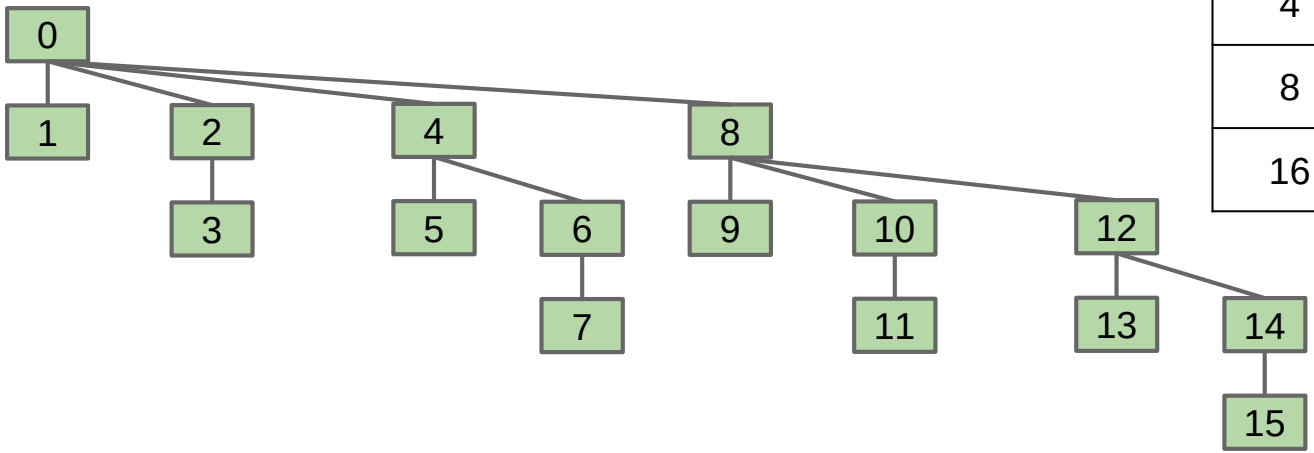
when N = 8, the tallest possible tree has height H = 3

Relationship between Weight and Height (9)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)

N	H
1	0
2	1
4	2
8	3
16	4

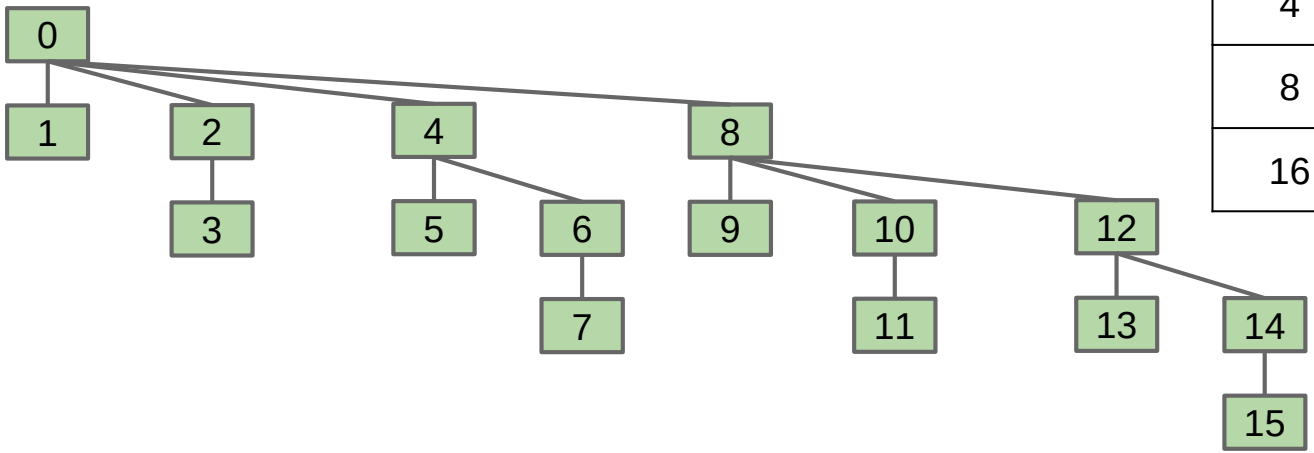
when N = 16, the tallest possible tree has height H = 4



Relationship between Weight and Height (10)

- We now illustrate the relationship between the weight (N, number of items) and the worst possible height (H)
  - the worst-case tree height H = O(log N)
  - since `union` and `isSameGroup` both depends on `find`, and `find` depends on the height, they are also O(log N)

N	H
1	0
2	1
4	2
8	3
16	4



Performance Summary

- Our fourth attempt has finally achieved a good performance!
  - logarithmic time is fast enough for practical problems

Implementation	constructor	union	isSameGroup
ListOfSetsUF	O(N)	O(N)	O(N)
QuickFind	O(N)	O(N)	O(1)
QuickUnion	O(N)	O(N)	O(N)
WeightedQuickUnion	**O(N)**	**O(log N)**	**O(log N)**

Thank you for your attention !

- In this lecture, you have learned:
  - to create an efficient Union Find data structure
  - to analyze its running time

- Please continue to **Lecture 8b**